

Backend programozás

Composer, static, Faker

Rostagni Csaba

2025. november 24.

Tartalom

- 1 Autoload
- 2 Composer
- 3 PHP Osztály szintű tulajdonságok és metódusok
- 4 Faker

Autoload - Osztályok automatikus betöltése

- Fárasztó az összes osztály beincludeolni.
- Amennyiben egy osztály hivatkozik egy másik osztályra, nekünk azt is be kellene tölteni úgy, hogy nem is tudunk róla (vagy sokat kellene nyomozni).
- Amennyiben az osztályainkat szabványosan hozzuk létre (pl PSR-4) használhatjuk az autoloadot.
- Az `spl_autoload_register()` függvény egy függvényt vár paraméterül, ami betölti a megfelelő osztályokat.
- A függvény betölti az osztályt, az első használatkor.
- Ezt a feladatot ma már a Composer látja el
<http://getcomposer.org>

Tartalom

- 1 Autoload
- 2 Composer
- 3 PHP Osztály szintű tulajdonságok és metódusok
- 4 Faker

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- composer.json - require
- Composer használata

Composer

- A Composer egy függőség kezelő rendszer PHP-hoz
- A <https://getcomposer.org> oldalon található meg
- A csomagokat a packagist.org oldalon keresi alapértelmezetten

Linkek:

- getcomposer.org
- packagist.org

Tartalom

2 Composer

- Bevezető
- **Felépítése**
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- composer.json - require
- Composer használata

Fájlok és mappák

```
/
├── .dockerignore
├── .gitignore
├── Dockerfile
├── composer.json
├── composer.lock
├── ma.php
├── src
└── vendor
```

- A **composer.json** tartalmazza többek között a kívánt csomagok listáját
- A **composer.lock** tartalmazza a ténylegesen kiválasztott verzióját a csomagoknak
- Az **ma.php** fájlban szeretnénk felhasználni a csomagkezelő által összegyűjtött csomagokat
- Az **src** mappa tartalmazza az általunk létrehozott kódot PSR-4 szabvány szerint
- A **vendor** mappa tartalmazza a külső csomagok kódját

Infó

A `composer.json` és a `composer.lock` is legyen a verziókezelve, de a `vendor` szerepeljen a `.gitignore` fájlban!

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- composer.json - require
- Composer használata

Dockerfile előkészítése a Composer-hez

CMD

```
FROM php:8.4.8-cli-alpine3.22

WORKDIR /app

RUN apk update && apk add --no-cache \
    libzip-dev zip p7zip \
    && docker-php-ext-install zip
```

- Telepítsük `zip` php extension függőségeit
 - Az alpine image minimális így, pár extra dolog szükséges:
 - `libzip-dev`, `zip`, `p7zip`
- Telepítsük `zip` extension-t
 - A `docker-php-ext-install` a hivatalos php image szkriptje

Composer "telepítése" docker image-be

CMD

```
docker pull composer/composer  
docker run --rm -it -v "$(pwd):/app" composer/composer install
```

- A Composer használható közvetlenül is
- Létezik belőle hivatalos docker image is `composer` néven

Dockerfile

```
COPY --from=composer:2.8.9 /usr/bin/composer /usr/bin/composer
```

- A `composer:2.8.9` image-ben megtalálható composer az `/usr/bin/composer` elérési útvonalon
- Másoljuk a készülő image `/usr/bin` mappájába a build során

Linkek:

- [Getting Started - Docker Image](#)

Dockerfile példa

Dockerfile

```
FROM php:8.4.8-cli-alpine3.22

ENV TZ=Europe/Budapest

WORKDIR /app

RUN adduser -D -u 1000 -s /bin/bash phpdocker \
    && apk update \
    && apk add --no-cache \
        bash \
        libzip-dev \
        zip \
        p7zip \
        tzdata \
    && docker-php-ext-install zip

COPY --from=composer:2.8.9 /usr/bin/composer /usr/bin/composer

USER phpdocker

ENTRYPOINT [ "/bin/bash" ]
```

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- composer.json - require
- Composer használata

Composer CLI használat

```
php composer.phar install
```

CMD

- Amennyiben a projekt gyökerében megtalálható a `composer.phar`

```
php composer install
```

CMD

- Amennyiben át lett nevezve a `composer.phar` simán `composer-re`

```
composer install
```

CMD

- Amennyiben a "bin" mappában található meg a `composer` fájl,
- **és** van rajta futtatási jog, **és** a php megfelelő verziója is be van konfigurálva, úgy kifejezetten egyszerűen használható

Linkek:

- Intro: globális telepítés - getcomposer.org

composer require

CMD

```
composer require monolog/monolog
```

- A kiválasztott csomagot hozzáadja a `composer.json` fájl `require` kulcsához
- A csomagot be is szerzi

CMD

```
composer require "mpdf/mpdf:^8.2"
```

- A kívánt verzió is meghatározható

composer install

PHP

```
composer install
```

- Ha már létezik a `composer.json` és a `composer.lock` fájl
- A `composer.lock` által meghatározott csomagok konkrét verzióját, és azok függőségeit is beszerzi
- Ha a `composer.lock` nem létezik, legenerálja

Infó

- A `vendor` mappa szerepeljen a `.gitignore` fájlban
- A repository friss klónozása után nincs `vendor` mappa, így `autoload.php` fájl sem
- A `composer install` mindezt legenerálja, a `composer.lock` fájlban található fix verziószámok alapján

composer update

```
composer update
```

PHP

- Már létezik a `composer.json`
- Megkeresi a legfrissebb verziót, majd futtatja a `composer install`-t
- Ha a `composer.lock` már létezik, abban is frissíti a verziókat

```
composer update monolog/monolog
```

CMD

- A csomagnév megadásával csak az adott csomag frissül

- Miért kellhet frissíteni?
 - Valamelyik összetevő biztonsági rést tartalmaz
 - Új funkció jelent meg

Infó

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
 - composer.json - require
 - Composer használata

SemVer - Szemantikus verziószámkezelés

MAJOR.MINOR.PATCH

- a **MAJOR** verziószámot, ha a változtatásaink eredményeként elveszítjük a kompatibilitást a korábbi verzióval,
- a **MINOR** verziószámot, ha olyan funkcionalitást adunk, ami megőrzi a visszafelé kompatibilitást, és
- a **PATCH** verziószámot, ha visszafelé kompatibilis hibajavításokat adunk hozzá.

Linkek:

- Szemantikus Verziószámozás 2.0.0 HU

Pontos verzió megadása

```
composer require vendor/package:1.4.2
```

CMD

- Csak egy konkrét verziót engedélyezünk.
- Nincs automatikus frissítés másik verzióra.
- Ez csak a `1.4.2` verziót fogja telepíteni.
- Frissítéshez kézzel kell módosítani

^ (caret/kalap) operátor

CMD

```
composer require vendor/package:^1.4
```

- Engedi a kompatibilis frissítéseket (ajánlott ezt választani).
- SemVer szerint: nem ugrik nagyobb MAJOR verzióra.
- Elfogadott tartomány: `>=1.4.0 <2.0.0`
- Patch és minor frissítések jöhetnek, de `2.0.0` már nem.
- Jelentése: „bármelyik kompatibilis 1.x verzió jó”.

Infó

Reguláris kifejezésekben a mintaillesztés elejét jelöli a ^ (caret/kalap) operátor, itt is valaminek a kezdetét jelöli

~ (tilde) operátor – patch frissítések

```
composer require vendor/package:~1.4.2
```

CMD

- A MAJOR és MINOR verzió fix
- Csak a PATCH módosulhat
- Elfogadott tartomány: `>=1.4.2 <1.5.0`
- `1.4.3` , `1.4.10` jó, de `1.5.0` már nem.

~ (tilde) operátor – minor szintig

```
composer require vendor/package:~1.4
```

CMD

- A MAJOR verzió fix
- Módosulhat a MINOR és a PATCH is
- Elfogadott tartomány: `>=1.4.0 <2.0.0`
- Hasonló a `^1.4`-hez, de bizonyos esetekben szigorúbb.

Wildcard (*) használata

```
composer require vendor/package:1.4.*
```

CMD

- A csillaggal (`*`) vakon behelyettesíti a kiányzó részt
- Elfogadott tartomány: `>=1.4.0 <1.5.0`
- Gyakorlatban hasonló, mint a `~1.4.0`.

Gyakorlati példák ismert csomagokkal

mPDF csomag

```
composer require mpdf/mpdf:^8.0
```

CMD

- Bármely 8.x verzió engedélyezett, de 9.0 már nem.

Whoops csomag

```
composer require filp/whoops:~2.0
```

CMD

- 2.0.x , 2.1.x , stb. – a pontos SemVer szabályok és a csomag verziói alapján.

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- **composer.json - require**
- Composer használata

composer.json - require

composer.json

```
{  
    "require": {  
        "monolog/monolog": "2.0.*",  
        "fakerphp/faker": "^1.23"  
    }  
}
```

- A `composer.json` a projekt gyökérmappájában legyen
- A `require` kulcs alatt a projekthez működéséhez szükséges függőségeket kell meghatározni
 - A kulcs a csomag megnevezése
 - Az érték pedig a verziószáma lesz (javasoltan a semantic versioning szerint)
 - A `monolog` verziója legalább 2.0, de a 2.1-t már nem érheti el
 - A `faker` verziója legalább 1.23, de a 2.0-t már nem érheti el

Linkek:

- Basic usage: The require key - [getcomposer.org](https://getcomposer.org/doc/04-schema.md#require)
- Schema: require - [getcomposer.org](https://getcomposer.org/doc/04-schema.md#require)

Tartalom

2 Composer

- Bevezető
- Felépítése
- Telepítés
- CLI
 - composer require
 - composer install
 - composer update
- Szemantikus verziószámozás
- composer.json - require
- **Composer használata**

Composer használata

```
composer require symfony/var-dumper
```

CMD

```
{  
    "require": {  
        "symfony/var-dumper": "^7.2"  
    }  
}
```

composer.json

```
include __DIR__ . "/vendor/autoload.php";  
use Symfony\Component\VarDumper\VarDumper;  
VarDumper::dump(['name' => 'Bence', 'age' => 19]);
```

start.php

```
^ array:2 [  
    "name" => "Bence"  
    "age" => 19  
]
```

Eredmény

Tartalom

- 1 Autoload
- 2 Composer
- 3 PHP Osztály szintű tulajdonságok és metódusok
- 4 Faker

Scope Resolution Operator

- Röviden dupla kettőspont (`::`)
- Héberül: "Paamayim Nekudotayim"
- Osztályon kívül az osztály nevét kell megadni.
- Osztályon belül a `self`, `parent` és `static` kulcsszavakkal használatos.
- PHP 5.3.0 óta használható változókkal is, de a változó neve nem lehet a fenti 3 kulcsszavak egyike sem.

Link:

- <https://www.php.net/manual/en/language.oop5.paamayim-nekudotayim.php>
- https://en.wiktionary.org/wiki/paamayim_nekudotayim

Konstansok osztályon belül

- Létrehozhatunk osztály szinten létező konstansokat. (nem példány szintű!)
- Az értékük nem változhat.
- A konstans deklarálásánál kell megadni az értékét! Csak konstans kifejezés lehet, nem lehet változó, vagy metódus hívás.
- Az osztályon belül a `self` kulcsszóval érhetjük el (vagy az osztály nevével).
- Az osztályon kívül az osztály nevével érhetjük el.
- A `const` kulcsszóval vezetjük be, de a dollár jelet nem tesszük ki!
- Nagy betűvel illik írni.
- Amennyiben a láthatóság nincs megadva automatikusan `public` lesz. (kompatibilitási okokból)

Link:

- <https://www.php.net/manual/en/language.oop5.constants.php>

Konstansok használata

PHP

```
class Kor{  
    public const PI = 3.14;  
  
    private $r;  
  
    public function __construct($r)  
    {  
        $this->r = $r;  
    }  
  
    public function getTerulet()  
    {  
        return $this->r ** 2 * self::PI;  
    }  
}  
  
$kor = new Kor(5);  
echo "PI: " . Kor::PI . "\n";  
echo "Terület:" . $kor->getTerulet();
```

Eredmény

PI: 3.14
Terület: 78.5

Osztály szintű metódusok

- A `static` kulcsszóval létrehozott metódusokból osztályonként egy közös létezik
- Az osztály példányosítása nélkül is használhatóak
- A `$this` kulcsszó nem használható benne
- Amennyiben a láthatóság nincs megadva, úgy automatikusan `public` lesz. (kompatibilitási okokból)
- Az UML osztálydiagram aláhúzással jelöli

Link:

- [urlhttps://www.php.net/manual/en/language.oop5.static.php](https://www.php.net/manual/en/language.oop5.static.php)

Osztály szintű metódusok

```

class Matek {
    static function osszead($a,$b)
    {
        return $a + $b;
    }

    public static function kivon($a,$b)
    {
        return $a - $b;
    }
}

$x = 5;
$y = 10;

$z = Matek::osszead($x,$y);
echo $z . "\n";

echo Matek::kivon($z,8);

```

PHP

C	Matek
	+osszead(\$a,\$b)
	+kivon(\$a,\$b)

- Az `osszead()` publikus, pedig nem mondtuk meg neki.
- Az `osszead()` és a `kivon()` is meghívható a `Matek` példányosítása nélkül.

15
7

Eredmény

Osztály szintű tulajdonságok

- A `static` kulcsszóval létrehozott változókból és metódusokból osztályonként 1 létezik.
- Az osztály példányosítása nélkül is használhatóak.
- A `$this` kulcsszó nem használható vele.
- Amennyiben a láthatóság nincs megadva automatikusan `public` lesz. (kompatibilitási okokból)
- Az UML osztálydiagram aláhúzással jelöli

Link:

- [urlhttps://www.php.net/manual/en/language.oop5.static.php](https://www.php.net/manual/en/language.oop5.static.php)

Osztály szintű tulajdonságok

PHP

```
class X {
    private static $szamlalo = 0;

    public function __construct()
    {
        self::$szamlalo++;
    }

    public static function eredmeny()
    {
        return self::$szamlalo;
    }
}

$x1 = new X();
$x2 = new X();
$x3 = new X();

echo X::eredmeny();
```

C	X
-	\$szamlalo = 0
+	__construct()
+	eredmeny()

- A **self** kulcsszóval hivatkozhatunk rá.
- A **this** kulcsszó nem használható!
- A dollár jelet ki kell rakni az osztály szintű változóra hivatkozásnál!

Eredmény

3

Tartalom

- 1 Autoload
- 2 Composer
- 3 PHP Osztály szintű tulajdonságok és metódusok
- 4 Faker

Tartalom

4 Faker

- Faker használat
- Szövegek
 - words()
- Számok
 - randomNumber()
 - randomFloat()
- Speciális
 - randomElement()
 - randomElements()
 - regexify()

Composer komplex példa: Faker

CMD

```
composer require fakerphp/faker
```

- A `composer require` letölti a meghatározott csomagot
- Verzió szám nélkül a legfrissebbet kapjuk meg
- Létrehozza az alábbiakat, ha még nem létezne
 - a `composer.json` fájlt
 - a `composer.lock` fájlt
 - a `vendor` mappát
 - a `vendor/autoload.php` fájlt

composer.json

```
{  
    "require": {  
        "fakerphp/faker": "^1.24"  
    },  
}
```


Composer komplex példa: Faker

PHP

```
include __DIR__ . "/vendor/autoload.php";

use Faker\Factory as KamuGyar;

$faker = KamuGyar::create('de_DE');

echo $faker->name() . PHP_EOL;
```

- A `__DIR__` egy konstans, ami az aktuális mappa elérését tartalmazza
- A `vendor/autoload.php` kezeli az osztályok betöltését
- A `use` miatt nem kell az osztályok teljes nevét végig gépelni
- A `create` osztály szintű metódus legyárt egy Faker objektumot

Eredmény

Frau Prof. Corina Meier

Tartalom

4 Faker

- Faker használat
- Szövegek
 - words()
- Számok
 - randomNumber()
 - randomFloat()
- Speciális
 - randomElement()
 - randomElements()
 - regexify()

words()

```
$faker->words(3)
```

PHP

- Megadott számú szót generál

```
['praesentium', 'possimus', 'modi']
```

Eredmény

```
$faker->words(3, true)
```

PHP

- Ha a második paramétere true, akkor tömb helyett szöveget ad vissza

```
'praesentium possimus modi'
```

Eredmény

Linkek:

- [words\(\)](#) - PHP faker

Tartalom

4 Faker

- Faker használat
- Szövegek
 - words()
- Számok
 - randomNumber()
 - randomFloat()
- Speciális
 - randomElement()
 - randomElements()
 - regexify()

Számok generálása

- `randomDigit()` Véletlen számjegy
- `randomNumber()` megadott számjegyből álló egész szám
- `numberBetween()` két szám közötti egész számot ad
- `randomFloat()` valós számot generál
- ...

randomNumber()

```
$faker->randomNumber(3)
```

PHP

- 0 és 3 közötti számjegyű számot generál

111 vagy 5 vagy 12 vagy 999

Eredmény

```
$faker->randomNumber(4, true)
```

PHP

- Pontosan 4 számjegyű számokat ad

4657 vagy 9999 vagy 6681

Eredmény

Linkek:

- [randomNumber\(\)](#) - PHP faker

randomFloat()

```
$faker->randomFloat(3)
```

PHP

- Az első paraméter a tizedesek számát határozza meg

Eredmény

837.622 vagy 112.956 vagy 985.112

```
$faker->randomFloat(3, 1.888, 22)
```

PHP

- A második paraméter a minimális érték
- A harmadik paraméter a maximális érték

Eredmény

1.888 vagy 16.334 vagy 21.765

Linkek:

- `randomFloat()` - PHP faker

Tartalom

4 Faker

- Faker használat
- Szövegek
 - words()
- Számok
 - randomNumber()
 - randomFloat()
- Speciális
 - randomElement()
 - randomElements()
 - regexify()

randomElement()

```
$faker->randomElement(['A', 'B', 'C', 'D'])
```

PHP

- Véletlenszerűen választ **egy** elemet a megadott tömbből

"C"

Eredmény

Linkek:

- [randomElement\(\)](#) - PHP faker

randomElements()

PHP

```
$faker->randomElements(['A', 'B', 'C', 'D'],2)
```

- Véletlenszerűen választ `$count` elemet a megadott tömbből
 - Alapértelmezetten 1 elemet
 - `null` esetén véletlen számú
 - Meghatározott számérték esetén pont annyit

Eredmény

```
['A', 'D']
```

Linkek:

- `randomElements()` - PHP faker

regexify()

PHP

```
$faker->regexify('[A-Z]{2}-[A-Z]{2}-[0-9]{3}')
```

- A regexnek megfelelő értéket generál
- A fenti példa nem tökéletes, de az új szabványos rendszámra hasonlító értékeket ad

Eredmény

AA-DX-729